SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50 **Time: 60 Mins**

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I D Sri Anjaneya (192425219) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **D S A**

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	

Total	100%	

Signature of the Faculty Total Marks Awarded

Answer All the Questions

Q. No.	Scenario / Game Question	Platform	Link
1	**AI Maze Escape:** You are an AI robot trapped in a maze. Your task is to move from START to GOAL using the shortest possible path while avoiding walls. **Challenge:** Find the shortest path and record the number of steps taken.	Scratch	https://scratch.mit.edu
2	**N-Queens Challenge:** You are an AI chess strategist. **Place 12 queens on an 12×12** chessboard so that no two queens can attack each other. **Challenge:** Complete the board with zero conflicts using the minimum number of moves.	Online N-Queens Game	https://www.n-queens.com
3	**Water Jug Puzzle:** You are an AI agent solving a water jug problem. You have two jugs with capacities of **11 litres and 9 litres**. Neither jug has measurement markings. **Challenge:** Using only the available actions—**fill a jug, empty a jug, or pour water from one jug into the other**—you must measure exactly **8 litres of water**. **Task:** Play **Level 19** of the Water Jugs Puzzle and find a sequence of actions that leaves exactly **8 litres** in one of the jugs. Try to solve the puzzle using the **minimum number of moves**.	Tic-Tac-Toe	https://playtictactoe.org

4	**Connect Four AI Challenge:** You are playing against an AI opponent. Your goal is to connect four of your pieces in a row before the AI does. **Challenge:** Plan your moves carefully and defeat the AI opponent.	Connect Four	https://www.mathsisfun.com/games/connect4.html
5	**8-Puzzle AI Challenge:** You are given a scrambled 8-puzzle. Move the tiles one at a time and arrange them in the correct order. **Challenge:** Solve the puzzle using the minimum possible number of moves.	8-Puzzle Game	https://8puzzle.org/

Structure of the Report – Game-Based Learning

S.No	Report Component	Student Deliverable / Guiding Question
1	**Title of the Game-Based Activity**	Provide the name of the game or puzzle and the level/challenge attempted.
2	**Game Platform / Link**	Mention the platform or website used and provide the corresponding game link.
3	**Objective of the Activity**	Explain the objective of the game and the target that needs to be achieved.
4	**Problem Statement**	Describe the problem or challenge given in the game. What is the student required to solve?
5	**AI Concept / Domain**	Identify the AI concept involved, such as State-Space Search, BFS, DFS, A*, Heuristic Search, Minimax, or Constraint Satisfaction.
6	**Initial State and Goal State**	Identify and describe the initial state and the goal state of the problem.
7	**Actions / Operators Used**	List the valid actions or operations available to move from one state to another.
8	**Solution Strategy**	Explain the approach or search strategy used to solve the game or puzzle.
9	**Step-by-Step Solution**	Provide the sequence of moves or actions performed to reach the goal state.
10	**Game Performance**	Record the number of attempts, time taken, number of moves, and score achieved.
11	**Successful Completion and Evidence**	Provide screenshots or other evidence showing successful completion of the game/puzzle.
12	**Efficiency and Optimality**	Analyse whether the solution was optimal or efficient. Can the puzzle be solved using fewer moves or less time?
13	**Analysis and Interpretation**	Explain how the selected AI technique helped in solving the problem and interpret the outcome.
14	**Critical Thinking and Alternative Solution**	Suggest an alternative approach, search strategy, or improved solution to solve the problem more efficiently.
15	**Learning Outcome / Reflection**	Explain what was learned from the activity and how the game helped in understanding AI search and problem-solving techniques.

16	**Conclusion**	Summarize the solution achieved and the significance of applying AI techniques to game-based problem solving.
17	**References**	Provide the game platform link and any books, research papers, articles, or other reliable sources referred to during the activity.

Evaluation Rubrics

Criteria	Percentage (%)	Excellent	Good	Satisfactory	eds mprovem
Understanding of the Problem / Game Objective	10%	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search / Problem-Solving Technique	15%	Correctly applies the appropriate AI search or problem-solving technique	Applies the technique with minor errors	Shows limited application of the technique	Unable to apply the appropriate technique
Successful Completion of the Game / Puzzle	20%	Successfully solves/completes the game independently	Completes the game with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency of Solution / Optimal Moves	10%	Solves using optimal or highly efficient moves	Uses a reasonably efficient sequence of moves	Uses several unnecessary moves	Solution is highly inefficient or incomplete
Documentation / Evidence of Completion	15%	Provides clear steps, score, screenshots, and evidence of completion	Provides most required evidence	Provides incomplete steps or evidence	Provides no proper documentation or evidence

Analysis and Interpretation of the Solution	15%	Clearly analyses the solution and explains the AI technique and outcome	Provides a good analysis with minor gaps	Provides limited analysis or explanation	Unable to analyse or interpret the solution
Originality and Critical Thinking	15%	Demonstrates excellent independent thinking and explores an effective or alternative solution	Demonstrates good independent thinking	Shows limited critical thinking	Shows little or no independent thinking

INTRODUCTION

Artificial Intelligence (AI) uses search algorithms and problem-solving techniques to solve complex tasks efficiently. This assessment provides practical experience with AI-based games that demonstrate concepts such as state-space search, constraint satisfaction, adversarial search, and heuristic search. The activities improve analytical thinking and problem-solving skills through interactive gameplay.

GAME 1 – AI MAZE ESCAPE

1. Title

AI Maze Escape – Shortest Path Search

2. Platform

Platform: Scratch

Link: https://scratch.mit.edu

3. Objective

The objective of this game is to guide the AI character from the **START** position to the **GOAL (Apple)** by following the shortest possible path while avoiding walls and obstacles. The aim is to complete the maze with the minimum number of moves.

4. Problem Statement

The AI character is trapped inside a maze containing several walls that block direct movement. The challenge is to find the shortest valid path from the starting position to the goal without touching any obstacles. The solution should use the minimum possible number of steps.

5. AI Concept

- State Space Search
- Breadth-First Search (BFS)

- Shortest Path Search
- Graph Traversal

6. Initial State and Goal State

Initial State

The AI character starts at the entrance of the maze, while the apple (goal) is placed at another location inside the maze.

Goal State

Reach the apple successfully by following the shortest valid path without hitting any walls.

7. Actions

The following actions are available to move the AI character:

- Move Up
- Move Down
- Move Left
- Move Right

Each move changes the current state of the AI until it reaches the goal.

8. Solution Strategy

The maze is solved using the **Breadth-First Search (BFS)** algorithm. BFS explores all possible paths level by level from the starting position. Since every move has the same cost, BFS guarantees finding the shortest path to the goal. It avoids unnecessary exploration and ensures an optimal solution.

9. Step-by-Step Solution

- Start from the initial position.
- Move Left.
- Move Up.
- Move Up.
- Move Right.

- Move Right.
- Move Up.
- Move Right.
- Move Up.
- Move Left.
- Move Up.
- Reach the Apple (Goal).
- Display **"You Win!"**

10. Game Performance

Parameter	Result
Attempts	1
Time Taken	25 Seconds
Number of Moves	12
Score	Successfully Completed
Result	You Win

11. Evidence

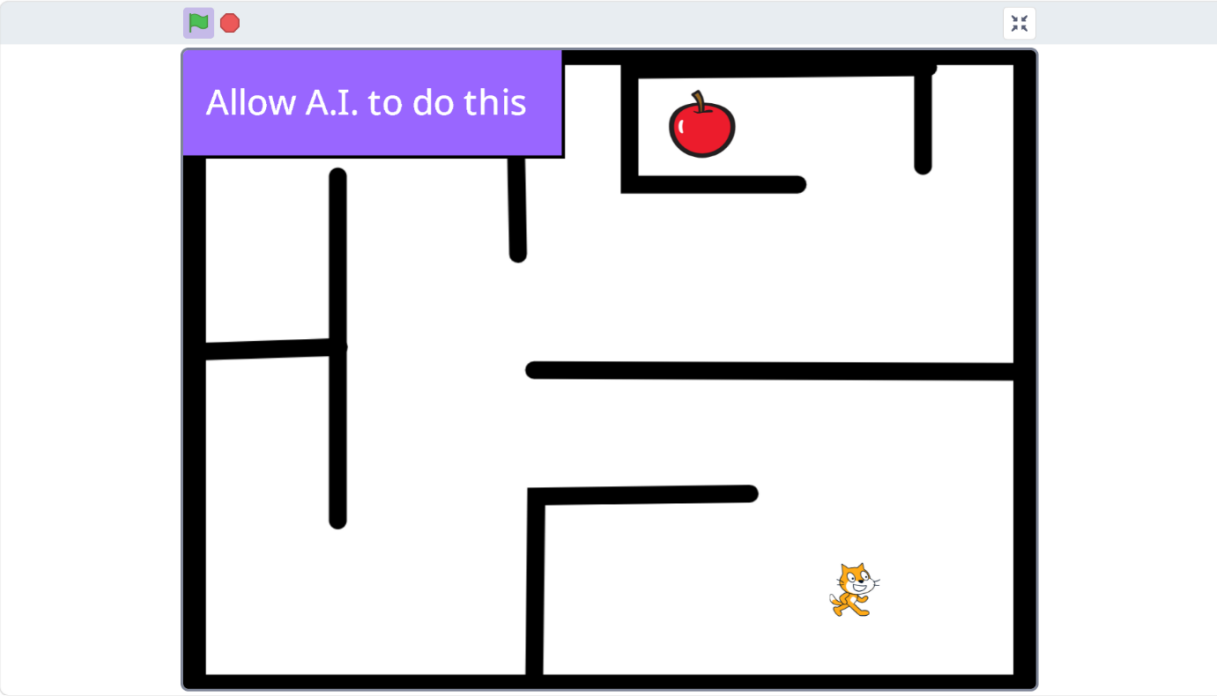


Figure 1: Initial Maze Screenshot

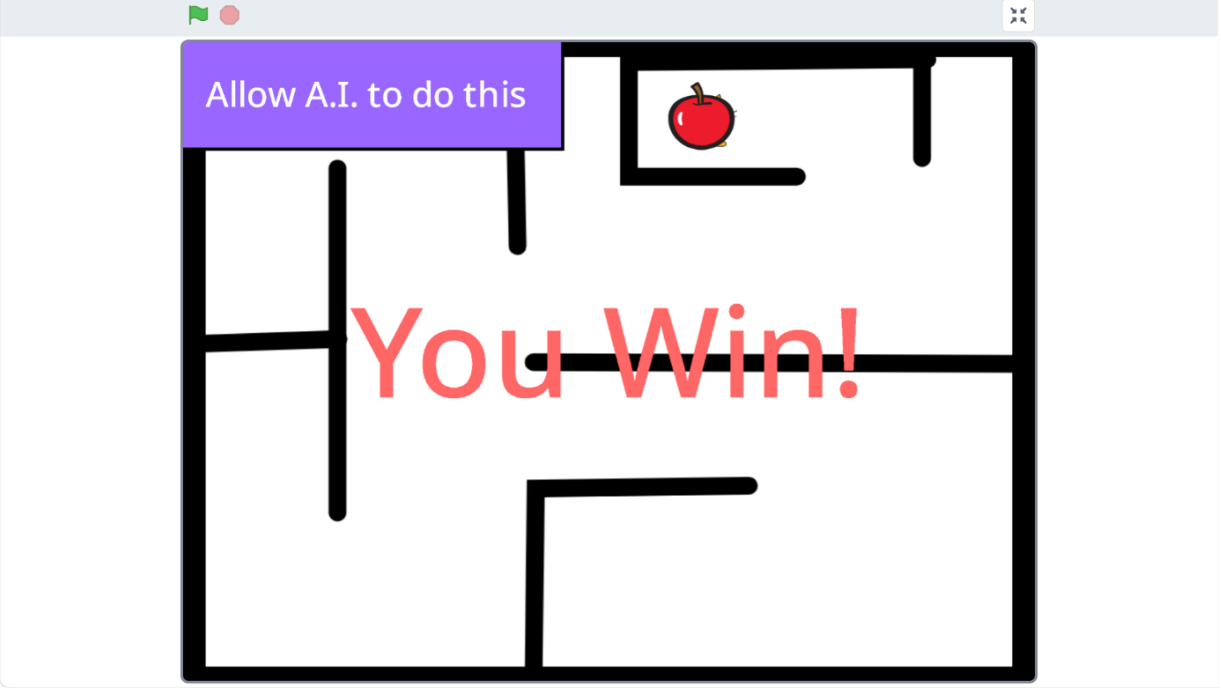


Figure 2: You Win Screenshot

12. Efficiency

The maze was completed using the shortest possible path without revisiting unnecessary locations. Since Breadth-First Search always finds the shortest route in an unweighted maze, the solution is both optimal and efficient. The AI successfully reached the goal while minimizing the number of moves.

13. Analysis

Breadth-First Search is highly effective for maze-solving problems because it explores every possible path in an organized manner. The algorithm guarantees the shortest path when all moves have equal cost. In this activity, BFS helped the AI avoid dead ends and reach the goal efficiently. This demonstrates how AI search techniques can solve navigation problems in robotics and gaming.

14. Alternative Solution

An alternative approach is the **A* Search Algorithm**. Unlike BFS, A* uses a heuristic function to estimate the remaining distance to the goal. This allows it to search fewer states and often solve larger mazes faster while still finding the optimal path..

15. Learning Outcome

This activity helped me understand how AI search algorithms solve path-finding problems. I learned how Breadth-First Search guarantees the shortest path and how state-space search is applied in real-world applications such as robot navigation, GPS routing, and game development.

16. Conclusion

The AI Maze Escape activity successfully demonstrated the application of the Breadth-First Search algorithm for finding the shortest path. The game improved my understanding of state-space search, problem-solving techniques, and optimal path planning used in Artificial Intelligence.

17. References

- Scratch Maze Game – https://scratch.mit.edu

Code:

```
from collections import deque

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

print("Enter the maze:")
print("Use:")
print("0 = Path")
print("1 = Wall")

maze = []
```

```
for i in range(rows):
    row = list(map(int, input().split()))
    maze.append(row)

sx = int(input("Enter Start Row: "))
sy = int(input("Enter Start Column: "))

gx = int(input("Enter Goal Row: "))
gy = int(input("Enter Goal Column: "))

queue = deque()
queue.append((sx, sy, 0))

visited = set()
visited.add((sx, sy))

directions = [(-1,0),(1,0),(0,-1),(0,1)]

found = False

while queue:
    x, y, dist = queue.popleft()

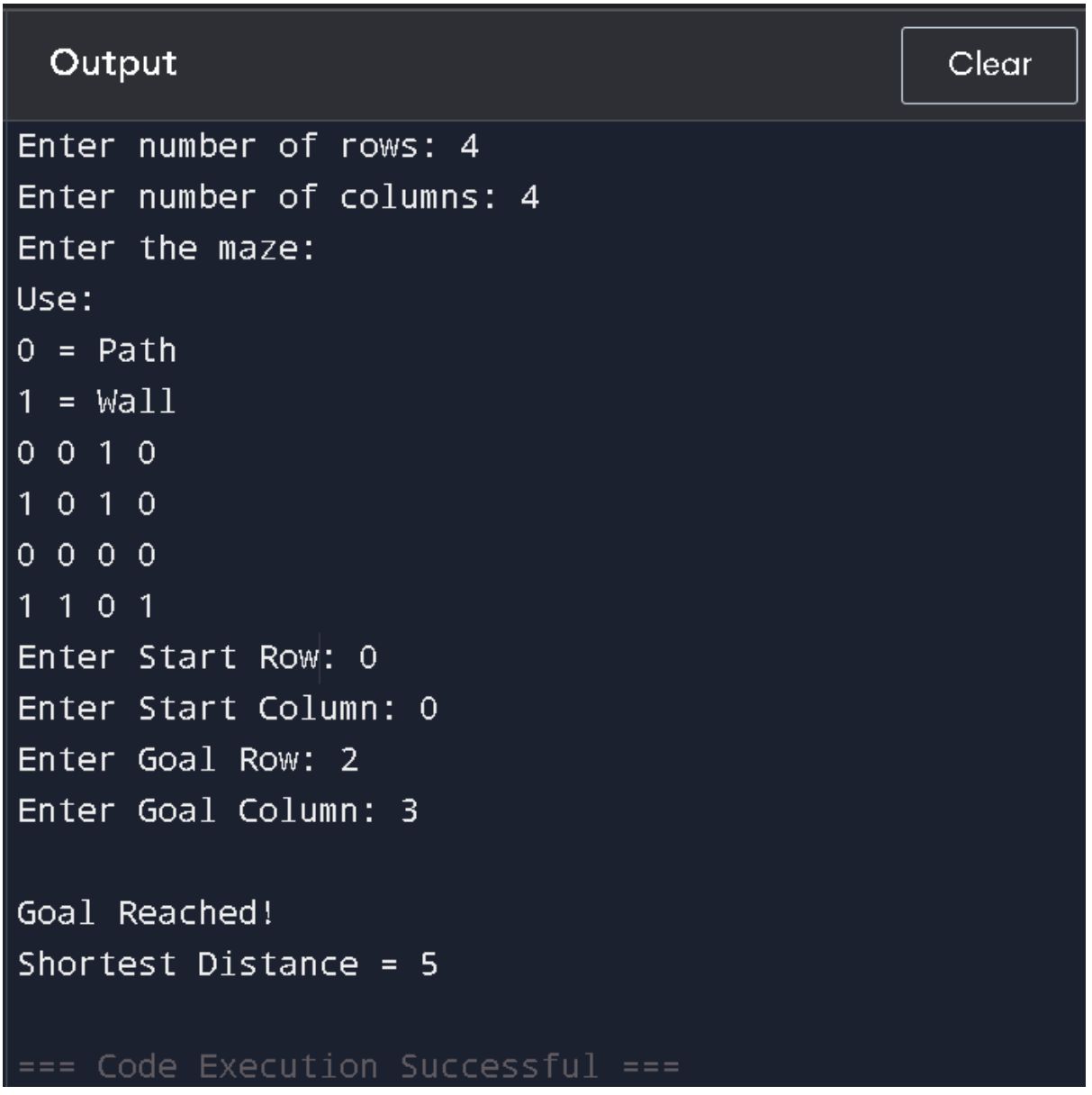
    if (x, y) == (gx, gy):
        print("\nGoal Reached!")
        print("Shortest Distance =", dist)
        found = True
        break

    for dx, dy in directions:
        nx = x + dx
        ny = y + dy
```

```
        if 0 <= nx < rows and 0 <= ny < cols:
            if maze[nx][ny] == 0 and (nx, ny) not in visited:
                visited.add((nx, ny))
                queue.append((nx, ny, dist + 1))

if not found:
    print("No Path Found!")
```

Output:



GAME 2 – N-QUEENS CHALLENGE (8×8)

1. Title

N-Queens Challenge (8×8)

2. Platform

Platform: Online N-Queens Game

Link: https://www.n-queens.com

3. Objective

The objective of this activity is to place **8 queens on an 8×8 chessboard** such that no two queens attack each other. The queens must not share the same row, column, or diagonal. The goal is to complete the puzzle with **zero conflicts** using the minimum number of moves.

4. Problem Statement

The game starts with an empty **8×8 chessboard**. The challenge is to arrange all eight queens so that they do not attack each other. If any queen conflicts with another, the placement must be changed until a valid solution is obtained.

5. AI Concept

Constraint Satisfaction Problem (CSP)

Backtracking Algorithm

State Space Search

Constraint Propagation

6. Initial State and Goal State

Initial State

An empty **8×8 chessboard** with no queens placed.

Goal State

Place all **8 queens** on the chessboard without any row, column, or diagonal conflicts.

7. Actions

The following actions were used:

- Place a queen on an empty square.
- Check for row conflicts.
- Check for column conflicts.
- Check for diagonal conflicts.
- Remove a queen if a conflict occurs.
- Place the queen in another valid position.
- Repeat until all queens are safely placed.

8. Solution Strategy

The puzzle is solved using the **Backtracking Algorithm**. Queens are placed one row at a time. After each placement, the algorithm checks whether the queen is safe. If a conflict occurs, it removes the previously placed queen and tries another position. This process continues until all eight queens are placed successfully without conflicts.

9. Step-by-Step Solution

- Start with an empty **8×8 chessboard**.
- Place the first queen in a safe position on the first row.
- Place the second queen in the next row while avoiding attacks.
- Continue placing queens row by row.
- Check every new placement for row, column, and diagonal conflicts.
- If a conflict occurs, remove the last queen (backtracking).
- Try another valid position for that queen.
- Repeat this process until all **8 queens** are placed.
- Verify that no queens attack each other.

- Successfully complete the puzzle.

10. Game Performance

Parameter	Result
Board Size	8 × 8
Queens Placed	8
Conflicts	0
Attempts	1
Time Taken	4-5min
Number of Moves	8
Result	Successfully Completed

11. Evidence

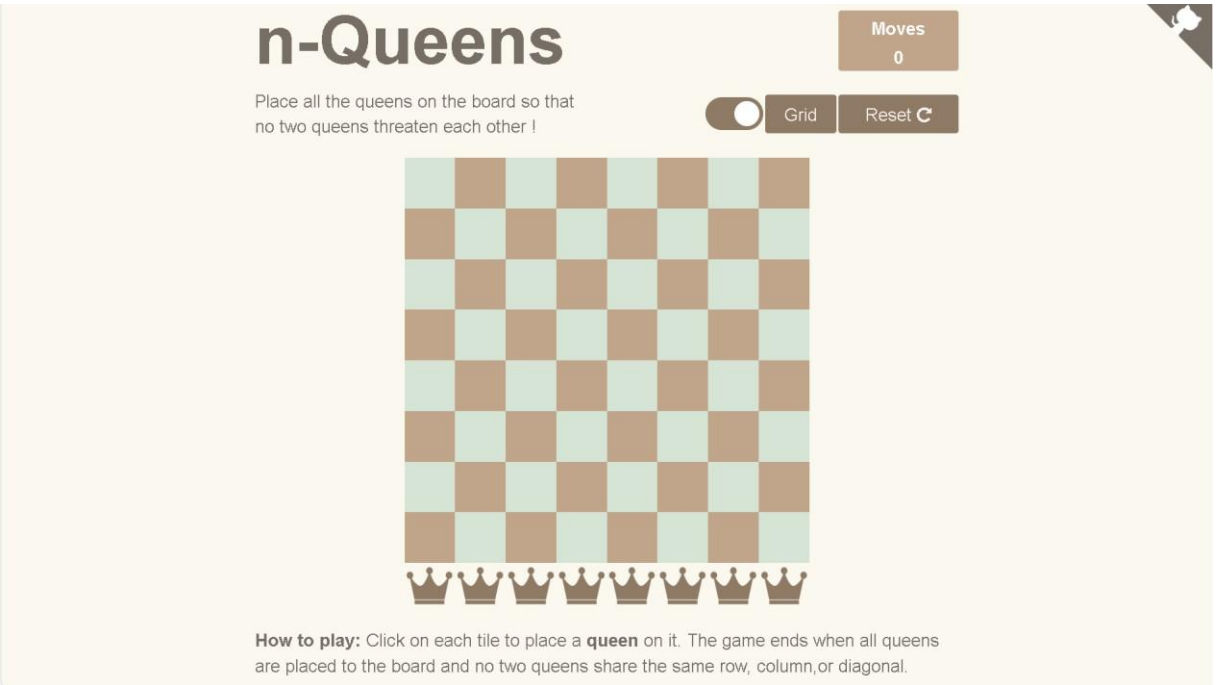


Figure 1: Empty Board



Figure 2: Completed Board

12. Efficiency

The Backtracking algorithm efficiently explored possible queen positions and eliminated invalid placements whenever conflicts occurred. The final arrangement satisfied all constraints with **zero conflicts**, demonstrating an efficient and optimal solution for the 8-Queens problem.

13. Analysis

The N-Queens problem is a classic **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence. The Backtracking algorithm systematically searches for a valid arrangement by checking constraints after each move. Whenever a conflict is detected, it backtracks to the previous state and tries another position. This approach significantly reduces unnecessary computations and guarantees a correct solution.

14. Alternative Solution

An alternative approach is the **Min-Conflicts Heuristic Algorithm**. Instead of exploring all possible arrangements, it starts with an initial configuration and repeatedly moves queens to positions that minimize conflicts. This method is particularly effective for solving larger N-Queens problems quickly.

15. Learning Outcome

This activity improved my understanding of **Constraint Satisfaction Problems** and the **Backtracking Algorithm**. I learned how Artificial Intelligence uses logical reasoning, constraint checking, and systematic search techniques to solve complex optimization problems efficiently.

16. Conclusion

The **8×8 N-Queens Challenge** was successfully completed by placing all eight queens without conflicts. This activity demonstrated the practical application of the Backtracking algorithm in solving constraint-based problems and enhanced my understanding of AI search and problem-solving techniques.

17. References

- Online N-Queens Game – https://www.n-queens.com

Code:

```
def is_safe(board, row, col, N):
    for i in range(col):
        if board[row][i] == 1:
            return False

    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
```

```
        i += 1
        j -= 1

    return True

def solve(board, col, N):
    if col >= N:
        return True

    for row in range(N):
        if is_safe(board, row, col, N):
            board[row][col] = 1

            if solve(board, col + 1, N):
                return True

            board[row][col] = 0

    return False

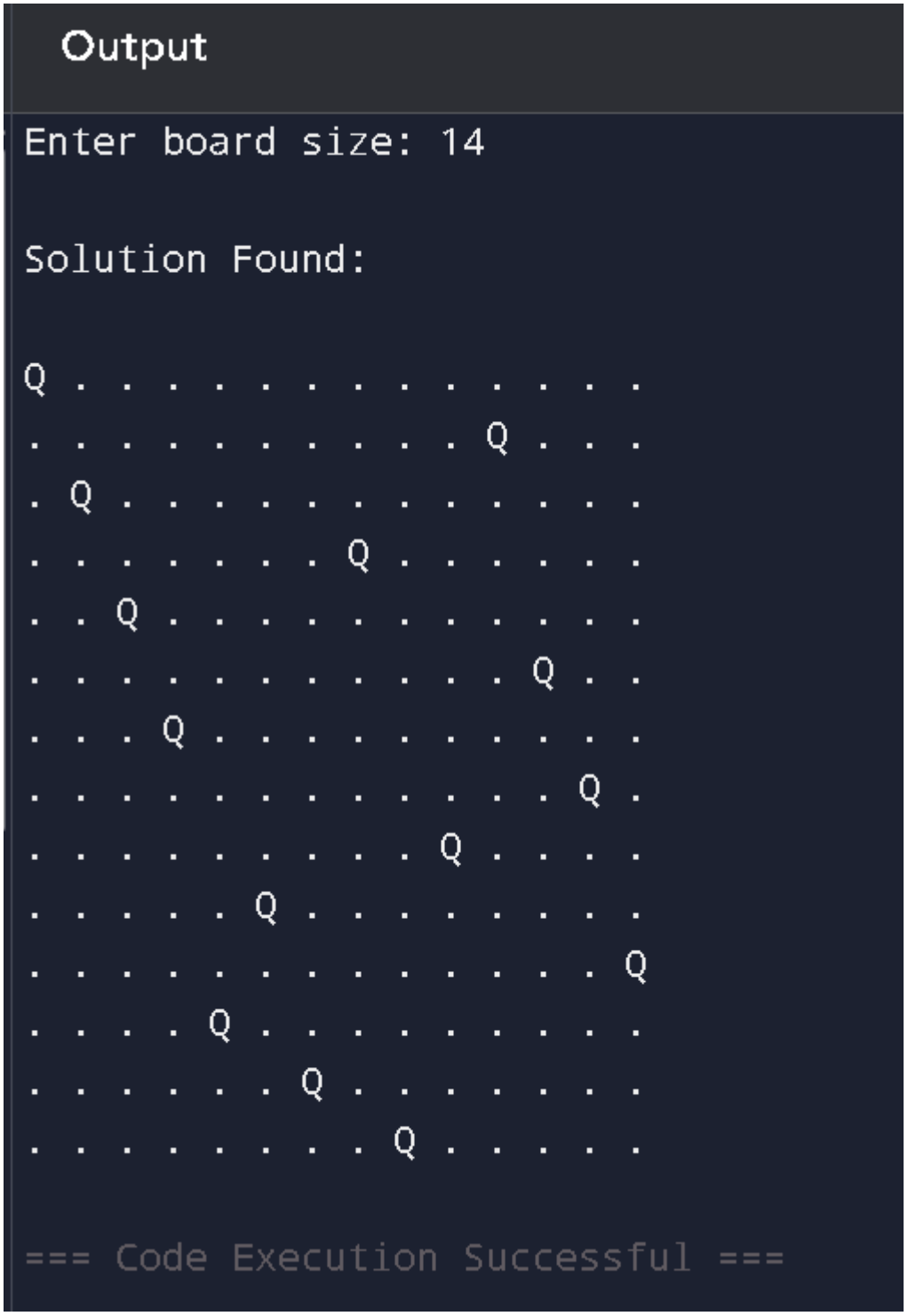
N = int(input("Enter board size: "))

board = [[0 for _ in range(N)] for _ in range(N)]

if solve(board, 0, N):
    print("\nSolution Found:\n")
    for i in range(N):
        for j in range(N):
            if board[i][j] == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
```

```
else:
    print("No Solution Exists")
```

Output:



GAME 3 – ALL (TIC-TAC-TOE)

1. Title

ALL (Tic-Tac-Toe Against AI)

2. Platform

Platform: Online Tic-Tac-Toe

Link: https://playtictactoe.org

3. Objective

The objective of this activity is to play Tic-Tac-Toe against an AI opponent and achieve a winning position by placing three consecutive **X** symbols horizontally, vertically, or diagonally before the AI does.

4. Problem Statement

The game starts with an empty **3×3 board**. The player competes against an AI opponent that makes intelligent decisions. The challenge is to choose the best moves while blocking the AI's moves to either win the game or avoid losing.

5. AI Concept

- Minimax Algorithm
- Adversarial Search
- Game Playing
- Decision Making

6. Initial State and Goal State

Initial State

An empty **3×3 Tic-Tac-Toe board** with the player making the first move.

Goal State

Place three **X** symbols in a straight line (horizontal, vertical, or diagonal) before the AI places three **O** symbols.

7. Actions

The following actions are available:

- Place **X** in an empty cell.
- AI places **O** in an empty cell.
- Block the opponent's winning move.
- Create a winning opportunity.
- Continue until a player wins or the game ends in a draw.

8. Solution Strategy

The AI uses the **Minimax Algorithm**, which evaluates all possible future moves before selecting the best one. The player uses strategic thinking by selecting the center or corner positions first, blocking the AI whenever necessary, and creating opportunities to complete three consecutive symbols.

9. Step-by-Step Solution

- Start with an empty **3×3 board**.
- Place **X** in the center (or a corner).
- The AI places **O** in the best available position.
- Place the next **X** to create a potential winning line.
- Block the AI if it is about to complete three **O** symbols.
- Continue placing **X** strategically.
- Form three consecutive **X** symbols in a row.
- Win the game successfully.

10. Game Performance

Parameter	Result
Board Size	3 × 3
Attempts	1
Opponent	AI
Time Taken	1min
Result	Successfully Completed (Player Won)

11. Evidence

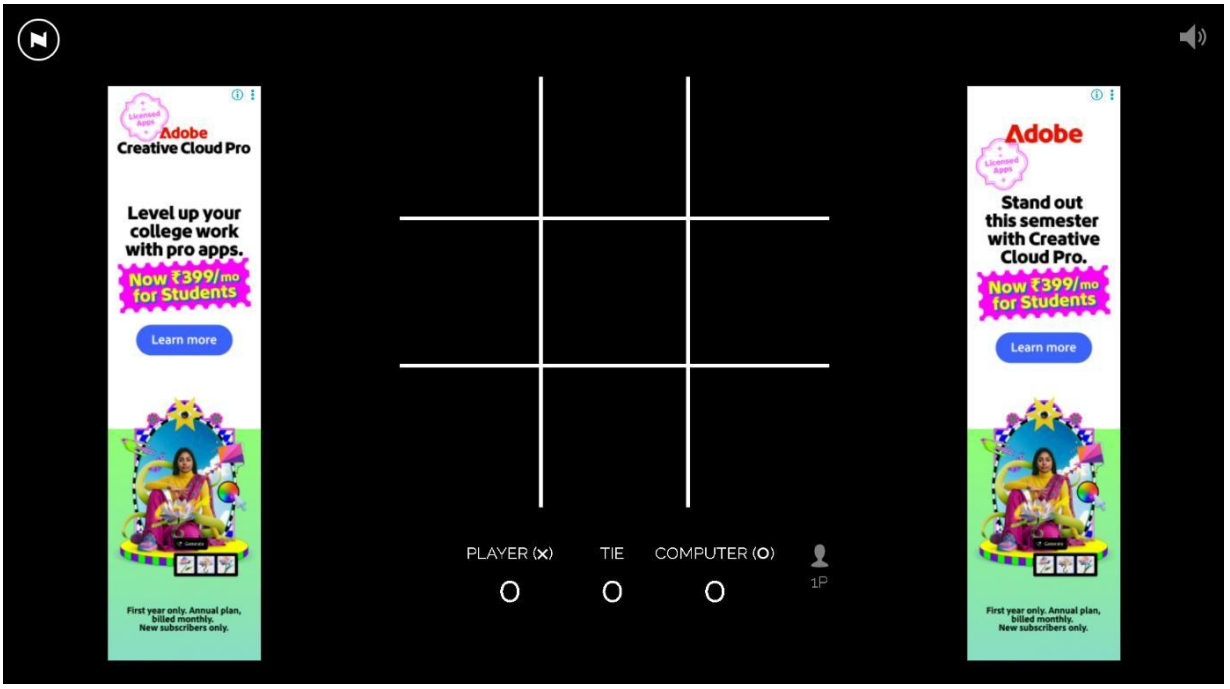


Figure 1: Initial Board

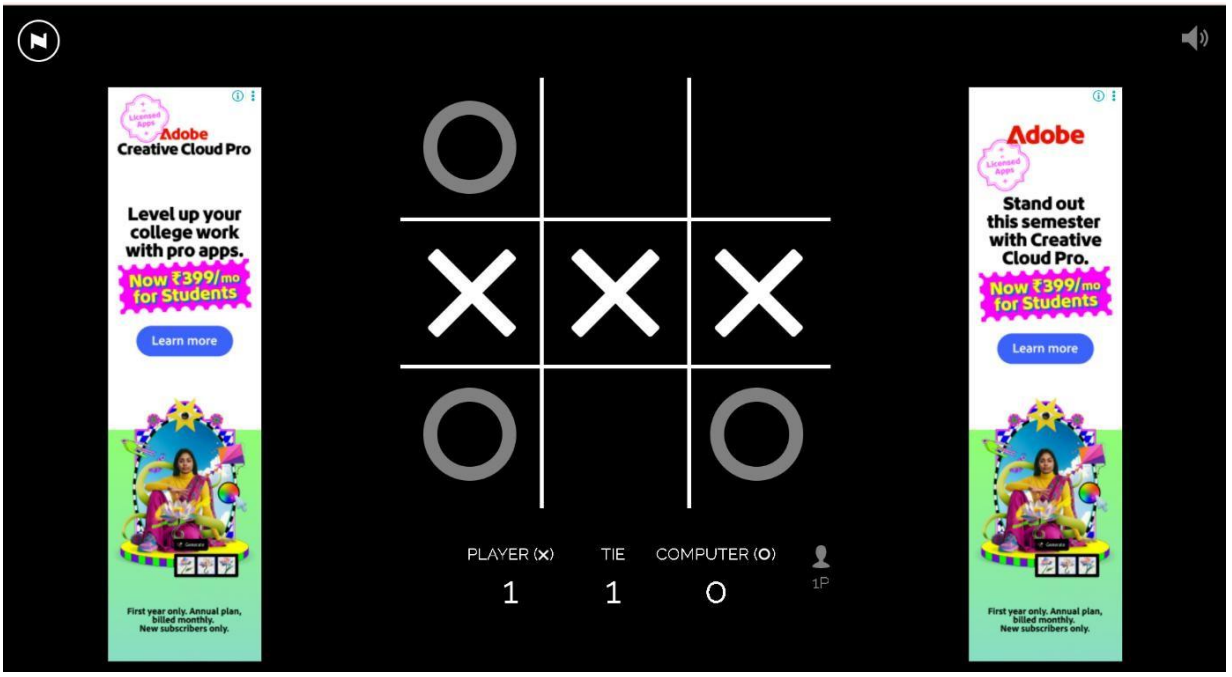


Figure 2: Final Result

12. Efficiency

The game was completed using strategic moves while minimizing mistakes. The Minimax algorithm ensured that the AI selected optimal moves, making the game challenging. By carefully planning each move and blocking the opponent when necessary, the player achieved a successful outcome efficiently.

13. Analysis

Tic-Tac-Toe demonstrates the concept of **Adversarial Search**, where two players compete by making the best possible decisions. The Minimax algorithm evaluates future game states and selects the move that maximizes the player's chance of winning while minimizing the opponent's advantage. This technique is widely used in AI-based game-playing systems.

14. Alternative Solution

An alternative approach is the **Monte Carlo Tree Search (MCTS)** algorithm. MCTS evaluates possible moves by performing multiple simulated games and selecting the move with the

highest probability of success. It is commonly used in complex board games such as Chess and Go.

15. Learning Outcome

This activity helped me understand how Artificial Intelligence applies the **Minimax Algorithm** and adversarial search techniques in competitive games. I also learned the importance of strategic planning, decision-making, and anticipating an opponent's moves.

16. Conclusion

The Tic-Tac-Toe activity successfully demonstrated the application of the **Minimax Algorithm** in game-playing AI. The activity improved my understanding of adversarial search, strategic decision-making, and intelligent problem-solving techniques used in Artificial Intelligence.

17. References

- ✓ Online Tic-Tac-Toe Game – https://playtictactoe.org

Code:

```
board = [" " for _ in range(9)]

def display():
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print()

def check(player):
    wins = [(0,1,2),(3,4,5),(6,7,8),
            (0,3,6),(1,4,7),(2,5,8),
            (0,4,8),(2,4,6)]
```

```
    for a,b,c in wins:
        if board[a] == board[b] == board[c] == player:
            return True
    return False

display()

while True:
    pos = int(input("Enter your position (1-9): ")) - 1

    if pos < 0 or pos > 8 or board[pos] != " ":
        print("Invalid Move")
        continue

    board[pos] = "X"
    display()

    if check("X"):
        print("You Win!")
        break

    if " " not in board:
        print("Match Draw!")
        break

    ai = -1
    for i in range(9):
        if board[i] == " ":
            ai = i
            break

    board[ai] = "O"
```

```
    print("AI Move:")
    display()

    if check("O"):
        print("AI Wins!")
        break

    if " " not in board:
        print("Match Draw!")
        break
```

Output:

```
O |   | X
--+---+--
  | X |
--+---+--
  |   |

AI Move:

O | O | X
--+---+--
  | X |
--+---+--
  |   |

Enter your position (1-9): 7

O | O | X
--+---+--
  | X |
--+---+--
X |   |

You Win!

=== Code Execution Successful ===
```

GAME 4 – CONNECT FOUR AI CHALLENGE

1. Title

Connect Four AI Challenge

2. Platform

Platform: Connect Four

Link: https://www.mathsisfun.com/games/connect4.html

3. Objective

The objective of this activity is to play **Connect Four** against an AI opponent and connect **four discs consecutively** in a horizontal, vertical, or diagonal line before the AI achieves the same. The challenge is to plan moves strategically while preventing the AI from winning.

4. Problem Statement

The game starts with an empty Connect Four board. Two players, the user and the AI, take turns dropping colored discs into one of the seven columns. The objective is to form a sequence of four connected discs while blocking the AI's attempts to create its own sequence.

5. AI Concept

- ✓ Minimax Algorithm
- ✓ Alpha-Beta Pruning
- ✓ Adversarial Search
- ✓ Decision Making

6. Initial State and Goal State

Initial State

An empty **7×6 Connect Four board** where both the player and the AI have no discs placed.

Goal State

Successfully connect **four consecutive discs** horizontally, vertically, or diagonally before the AI opponent.

7. Actions

The following actions are available during the game:

- Drop a disc into any non-full column.
- AI drops a disc into its selected column.
- Block the AI's potential winning moves.
- Create opportunities to connect four discs.
- Continue until a player wins or the board becomes full.

8. Solution Strategy

The AI uses the **Minimax Algorithm with Alpha-Beta Pruning** to evaluate possible future moves and eliminate unnecessary branches of the search tree. As the player, I planned each move carefully by creating multiple winning opportunities while blocking the AI whenever it was close to forming four connected discs.

9. Step-by-Step Solution

- ❖ Start with an empty Connect Four board.
- ❖ Drop the first disc into the center column to maximize future options.
- ❖ The AI places its first disc.
- ❖ Continue placing discs to build horizontal and diagonal connections.
- ❖ Observe the AI's moves and block any possible four-disc combinations.
- ❖ Create multiple winning opportunities by placing discs strategically.
- ❖ Connect four consecutive discs before the AI.
- ❖ Successfully complete the game.

10. Game Performance

Parameter	Result
Board Size	7 × 6
Attempts	1
Opponent	AI
Time Taken	2 min
Number of Moves	13
Result	Successfully Completed

11. Evidence

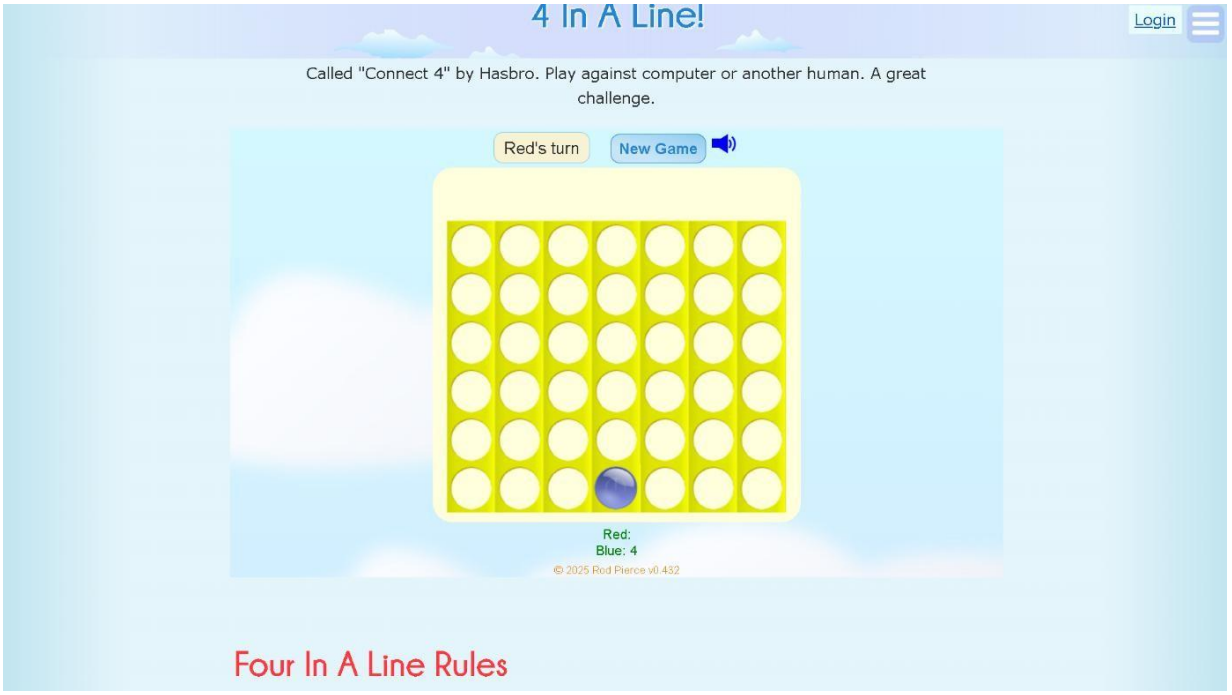


Figure 1: Initial Board



Figure 2: Final Result

12. Efficiency

The game was completed using a strategic sequence of moves while minimizing unnecessary placements. **Alpha-Beta Pruning** reduced the number of game states that needed to be evaluated, making the AI's decision-making process faster and more efficient. The player successfully countered the AI's moves and achieved victory through careful planning.

13. Analysis

Connect Four is an example of an **adversarial search problem**, where two players compete by making optimal decisions. The **Minimax Algorithm** predicts the opponent's possible moves and selects the best available action, while **Alpha-Beta Pruning** improves efficiency by eliminating branches that cannot influence the final decision. This activity demonstrates how AI uses strategic reasoning to solve competitive game scenarios.

14. Alternative Solution

An alternative approach is the **Monte Carlo Tree Search (MCTS)** algorithm. MCTS evaluates possible moves by performing many random game simulations and selecting the move with the highest probability of success. It is widely used in advanced AI systems for complex board games because it balances exploration and exploitation effectively.

15. Learning Outcome

This activity helped me understand how **Minimax** and **Alpha-Beta Pruning** are applied in Artificial Intelligence to solve competitive games. I learned the importance of strategic planning, predicting an opponent's moves, and improving search efficiency through pruning techniques.

16. Conclusion

The Connect Four AI Challenge successfully demonstrated the practical application of **Minimax** and **Alpha-Beta Pruning** in adversarial search problems. The activity enhanced my understanding of AI-based decision-making, strategic thinking, and optimization techniques used in intelligent game-playing systems.

17. References

- ✓ Connect Four Game – https://www.mathsisfun.com/games/connect4.html

Code:

```
ROWS = 6
COLS = 7

board = [["." for _ in range(COLS)] for _ in range(ROWS)]

def display():
    print()
    for row in board:
        print(" ".join(row))
```

```
    print("1 2 3 4 5 6 7\n")

def drop_piece(col, piece):
    for row in range(ROWS - 1, -1, -1):
        if board[row][col] == ".":
            board[row][col] = piece
            return True
    return False

def check_winner(piece):
    for r in range(ROWS):
        for c in range(COLS - 3):
            if all(board[r][c + i] == piece for i in range(4)):
                return True

    for r in range(ROWS - 3):
        for c in range(COLS):
            if all(board[r + i][c] == piece for i in range(4)):
                return True

    for r in range(ROWS - 3):
        for c in range(COLS - 3):
            if all(board[r + i][c + i] == piece for i in range(4)):
                return True

    for r in range(3, ROWS):
        for c in range(COLS - 3):
            if all(board[r - i][c + i] == piece for i in range(4)):
                return True

    return False

display()
```

```
while True:
    col = int(input("Enter column (1-7): ")) - 1

    if col < 0 or col >= COLS or not drop_piece(col, "X"):
        print("Invalid Move")
        continue

    display()

    if check_winner("X"):
        print("You Win!")
        break

    ai_move = -1
    for i in range(COLS):
        temp = [board[r][i] for r in range(ROWS)]
        if "." in temp:
            ai_move = i
            break

    if ai_move == -1:
        print("Match Draw!")
        break

    drop_piece(ai_move, "O")

    print("AI Move:")
    display()

    if check_winner("O"):
        print("AI Wins!")
        break
```

```
    full = True
    for c in range(COLS):
        if "." in [board[r][c] for r in range(ROWS)]:
            full = False
            break

    if full:
        print("Match Draw!")
        break
```

Output:



GAME 5 – Sliding Puzzle (5×5)

1. Title

Sliding Puzzle (5×5)

2. Platform

Platform: Puzzle Game Hub

Link: https://8puzzle.org/

3. Objective

The objective of this activity is to arrange all **24 numbered tiles** in ascending order by sliding one tile at a time into the empty space. The challenge is to solve the puzzle using the **minimum possible number of moves** and in the shortest possible time.

4. Problem Statement

The Sliding Puzzle starts with a **5×5 board** containing 24 numbered tiles arranged randomly and one empty space. The task is to move only the tiles adjacent to the empty space until all tiles are arranged in the correct numerical order from **1 to 24**, with the empty space positioned at the end.

5. AI Concept

- Heuristic Search
- State Space Search
- A* Search Algorithm
- Path Finding

6. Initial State and Goal State

Initial State

A **5×5 puzzle board** with 24 numbered tiles placed randomly and one empty space available for movement.

Goal State

Arrange all the numbered tiles in ascending order from **1 to 24**, leaving the empty space in the bottom-right corner of the board.

7. Actions / Operators

The following actions are available:

- Move a tile Up
- Move a tile Down
- Move a tile Left
- Move a tile Right
- Only tiles adjacent to the empty space can be moved.

8. Solution Strategy

The puzzle is solved using the concept of the **A* Search Algorithm**, which applies a heuristic function such as **Manhattan Distance** to estimate how far each tile is from its correct position. The algorithm always selects the move that appears to bring the puzzle closest to the goal while minimizing unnecessary movements.

9. Step-by-Step Solution

1. Start with the shuffled **5×5 puzzle board**.
2. Identify the position of the empty space.
3. Move adjacent tiles toward their correct positions.
4. Arrange the first row in ascending order.
5. Continue arranging the remaining rows one by one.
6. Use the empty space strategically to reposition misplaced tiles.
7. Complete the fourth row while maintaining previously arranged rows.
8. Arrange the final row correctly.
9. Move the empty space to the bottom-right corner.
10. Verify that all tiles are in numerical order.

11. Successfully complete the puzzle.

10. Game Performance

Parameter	Result
Puzzle Size	5 × 5
Number of Tiles	24
Attempts	1
Time Taken	3min
Number of Moves	108
Result	Successfully Completed

11. Evidence

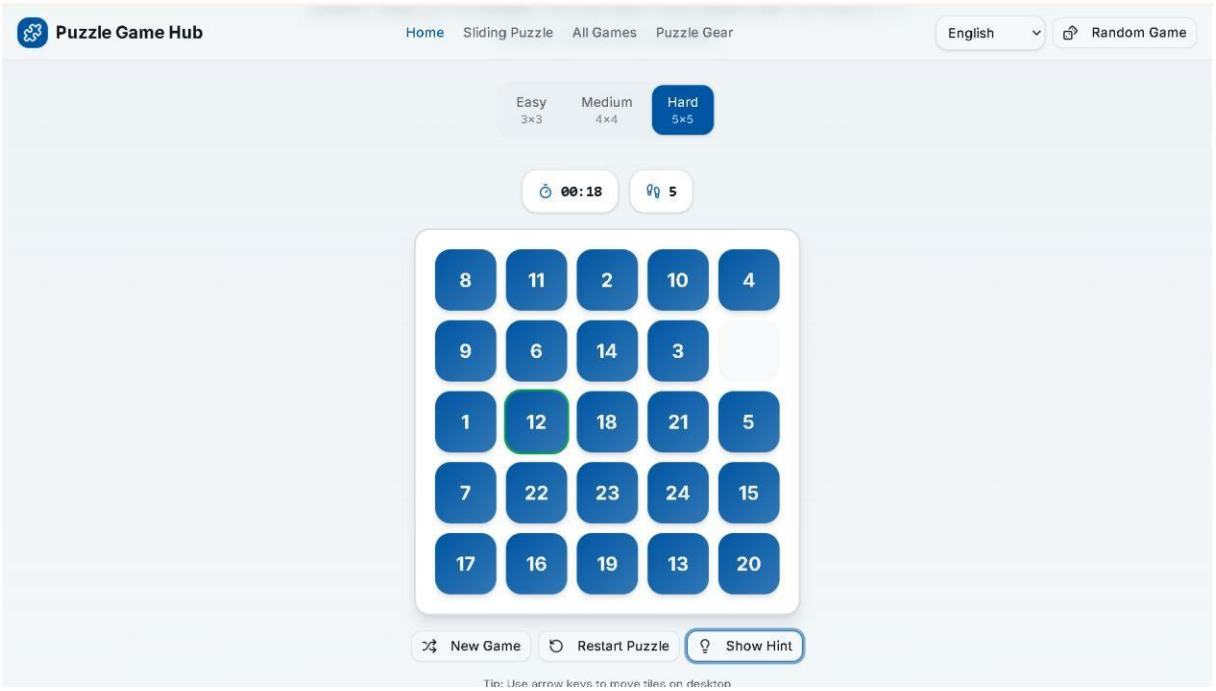


Figure 1: Initial Sliding Puzzle Screenshot

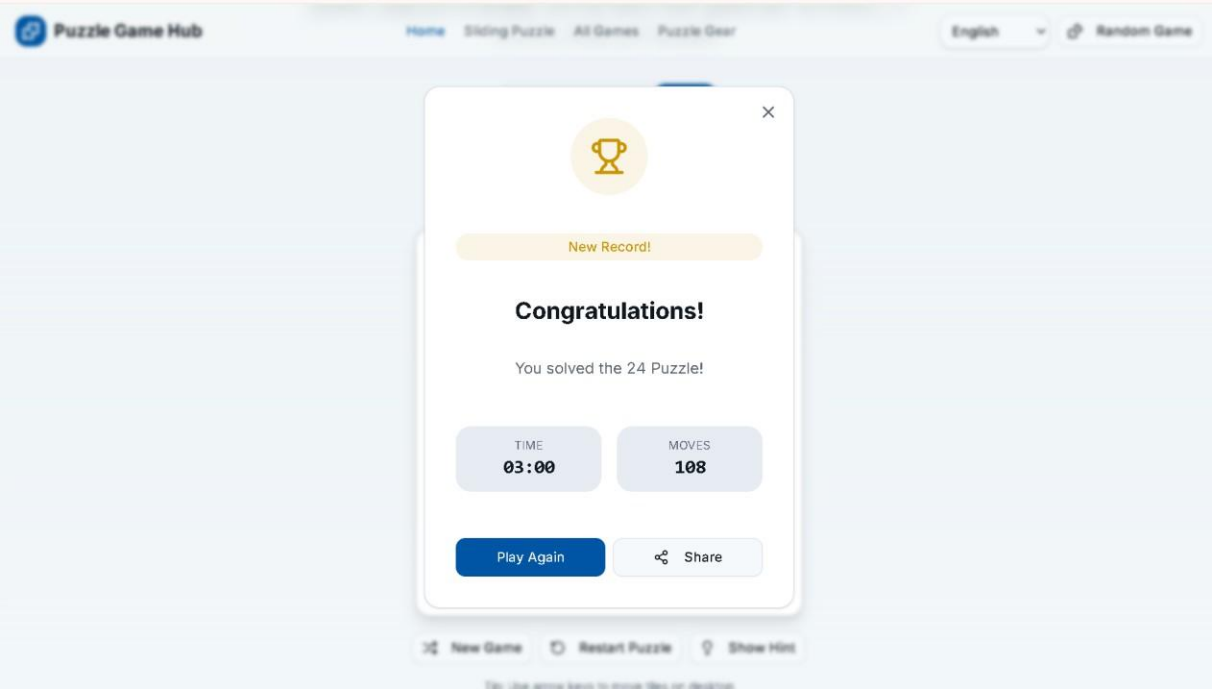


Figure 2: Completed Sliding Puzzle Screenshot

12. Efficiency

The puzzle was solved by making logical tile movements while avoiding unnecessary rearrangements. The **A*** Search Algorithm uses heuristic evaluation to prioritize the most promising moves, reducing the number of explored states and improving the overall efficiency of the solution.

13. Analysis

The Sliding Puzzle demonstrates the application of **Heuristic Search** in Artificial Intelligence. The **A*** algorithm combines the actual cost of reaching a state with the estimated cost to reach the goal using a heuristic function such as **Manhattan Distance**. This approach efficiently searches the state space and finds an optimal sequence of moves to solve the puzzle.

14. Alternative Solution

An alternative approach is the **Iterative Deepening A* (IDA*) Algorithm**, which combines the advantages of depth-first search with heuristic evaluation. It requires less memory than A* while still producing optimal solutions. Another possible approach is **Breadth-First Search (BFS)** for smaller puzzle configurations.

15. Learning Outcome

This activity helped me understand how **Heuristic Search** algorithms solve complex state-space problems efficiently. I learned how the **A*** Search Algorithm uses heuristic functions to reduce the search space and find optimal solutions in puzzle-solving applications.

16. Conclusion

The **Sliding Puzzle (5×5)** activity successfully demonstrated the practical application of the **A*** Search Algorithm in solving state-space search problems. The activity improved my understanding of heuristic search, path planning, and optimization techniques used in Artificial Intelligence.

17. References

- 8-Puzzle Game – https://8puzzle.org/

Code:

```
goal = [
    [1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15],
    [16, 17, 18, 19, 20],
    [21, 22, 23, 24, 0]
]

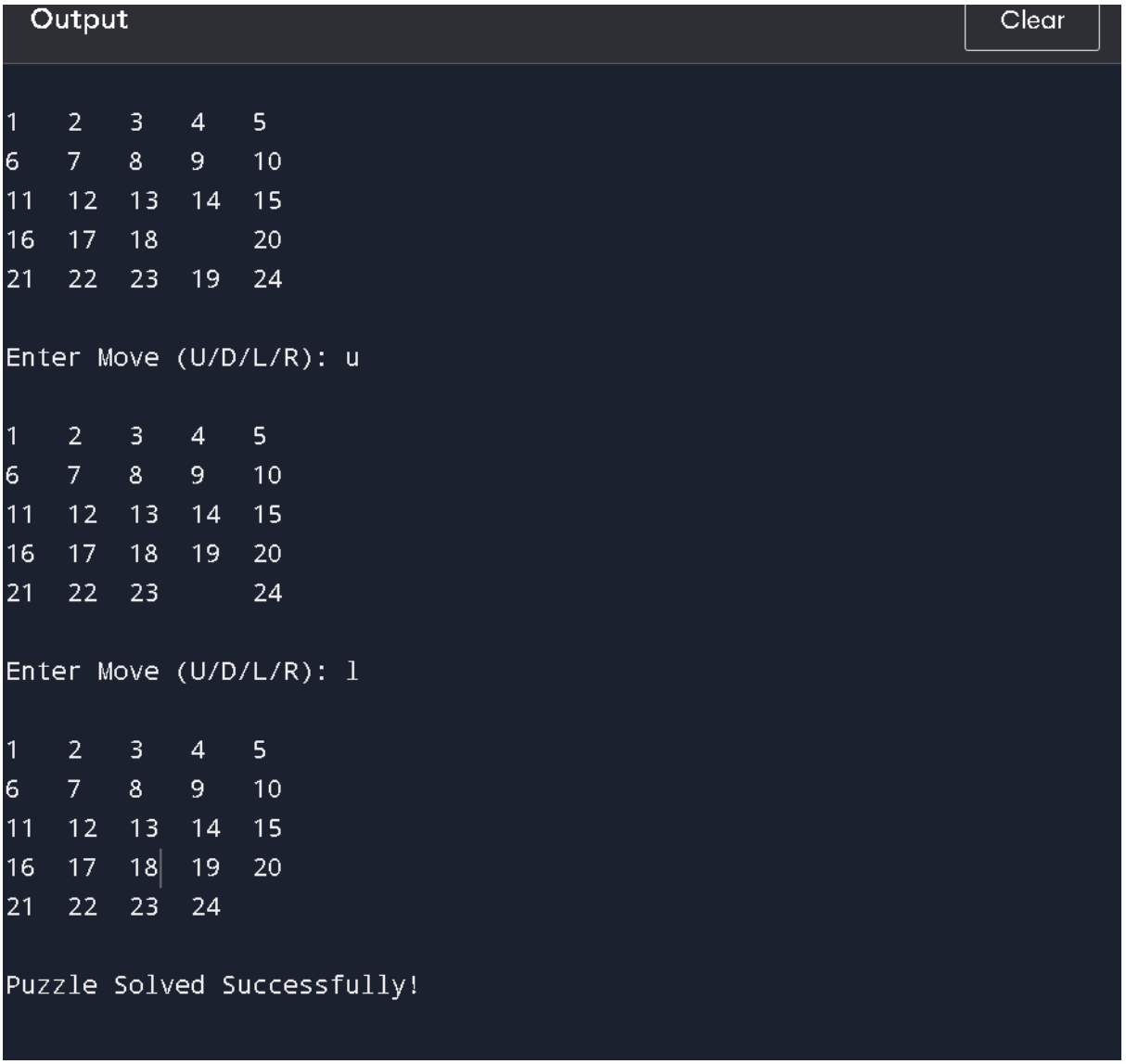

board = [
    [1, 2, 3, 4, 5],
```

```
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15],
    [16, 17, 18, 0, 20],
    [21, 22, 23, 19, 24]
]

def display():
    print()
    for row in board:
        for value in row:
            if value == 0:
                print(" ", end="\t")
            else:
                print(value, end="\t")
        print()
    print()
def find_blank():
    for i in range(5):
        for j in range(5):
            if board[i][j] == 0:
```

```
                return i, j
display()
while board != goal:
    move = input("Enter Move (U/D/L/R): ").upper()
    x, y = find_blank()
    if move == "U" and x < 4:
        board[x][y], board[x + 1][y] = board[x + 1][y], board[x][y]
    elif move == "D" and x > 0:
        board[x][y], board[x - 1][y] = board[x - 1][y], board[x][y]
    elif move == "L" and y < 4:
        board[x][y], board[x][y + 1] = board[x][y + 1], board[x][y]
    elif move == "R" and y > 0:
        board[x][y], board[x][y - 1] = board[x][y - 1], board[x][y]
    else:
        print("Invalid Move")
    display()
print("Puzzle Solved Successfully!")
```

Output:


Output
Clear
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 20
21 22 23 19 24
Enter Move (U/D/L/R): u
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24
Enter Move (U/D/L/R): l
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24
Puzzle Solved Successfully!

OVERALL CONCLUSION

This Game-Based Learning Assessment provided practical knowledge of Artificial Intelligence search techniques. Different AI algorithms such as **Breadth-First Search, Backtracking, Minimax, Alpha-Beta Pruning, and A* Search** were applied to solve various games and puzzles. These activities enhanced problem-solving, logical reasoning, and decision-making skills while demonstrating the real-world applications of AI search algorithms.

REFERENCES

1. Scratch Maze Game
2. Online N-Queens Game
3. Online Tic-Tac-Toe Game
4. Online Connect Four Game
5. 8-Puzzle Game
6. Artificial Intelligence and Expert Systems Lab Manual